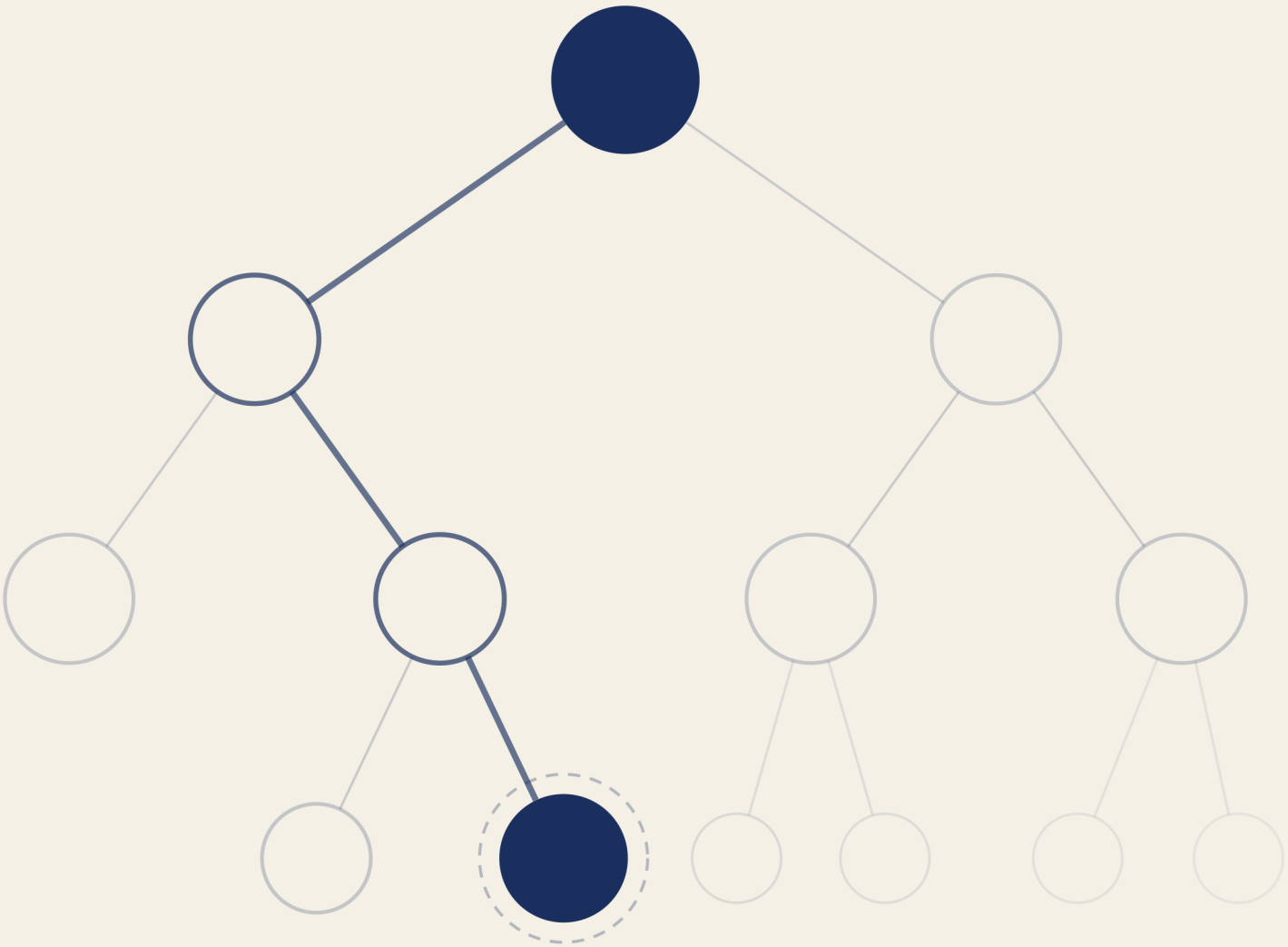
Practical Problem Solving

From Computational Thinking to Programs with Python



Igor Benav

Practical Problem Solving

From Computational Thinking to Programs with Python

Igor Benav

Contents

Preface

Beginner programming books usually teach you a language. They walk through syntax, show you features, and hope problem-solving follows, but it usually doesn't. You finish the book knowing what a `for` loop is but not necessarily when to use one. You can read code but can't design a solution from scratch.

This book reverses the order. We start with problems and introduce Python as the tool to solve them, with every concept existing because a problem demands it, not because it's the next item in a language tour. The goal isn't to memorize syntax, it's to think clearly enough about a problem that the code follows naturally.

This is more important now than it was some years ago. AI tools like ChatGPT can generate working code from a prompt, which means the bottleneck in programming has shifted. Syntax is no longer the hard part - knowing what to ask for, evaluating whether the result is correct, and understanding why an approach fails at scale: that's the hard part. This book builds those skills. Using AI to help write code is perfectly fine, but you should be able to judge whether a solution is good or broken, and know what to do about it.

About This Book

The book is opinionated. We use Python throughout, uv for package management, and the REPL for quick experiments. We're not surveying every language feature or teaching you the "best" way, we're teaching you one coherent path from "I've never programmed" to "I can solve real problems with code," so you have solid ground to stand on when you explore other directions later.

Computer science concepts (algorithms, abstraction, decomposition) aren't treated as background reading, they're introduced when the problem at hand requires them. You'll understand binary search not because the chapter is titled "Algorithms" but because you're trying to find a name in a phone book and the obvious approach is too slow.

What You'll Build

The book follows a single thread: a receipt printer that starts simple and grows more capable as you learn new concepts. It's not a toy example. It's a program that hits a real wall at the end of every chapter, and the next chapter's concepts are what you need to break through.

In Chapter 2, you build the first version: collecting input, storing it in variables, and printing formatted text. It works, but it can't calculate a total because `input()` returns strings and strings can't do math. That wall motivates Chapter 3, where you learn about types and conversion. Now the receipt computes totals, applies tax, and formats currency.

But it handles exactly three items, hardcoded. That motivates Chapter 4 (conditionals: skip items with zero quantity, apply discounts based on the subtotal) and Chapter 5 (loops: handle any number of items without duplicating code). Each chapter solves the previous chapter's limitation and reveals the next one.

By the end of the book, you'll have built programs that take input, make decisions, repeat actions, work with structured data, and solve problems you couldn't have approached at the start. More importantly, you'll have a way of thinking about problems that works whether you're writing Python, learning another language, or telling an AI what to build.

How to Use This Book

The chapters are meant to be read in order, each one builds on the previous. If you skip types, the chapter on conditionals won't make sense. If you skip conditionals, you'll be lost when loops arrive.

Each chapter opens with a problem you can't solve yet. By the end, you can. The concepts exist because the problem demands them: we understand the motivation, see the concept expressed in Python, and then apply it in a hands-on project that extends the running receipt printer (or a related program). This structure was deliberate, it means you always know *why* you're learning something before you learn *how*.

Type the code yourself, don't copy and paste. The errors you make and fix along the way are part of learning. Change things, break things, see what happens.

When you get stuck, resist asking AI for the solution. Ask for a hint if you need to, but do the thinking yourself - that's the part you CAN'T delegate. Struggling is normal, it's also how you actually learn, rather than feeling like you're learning.

One recommendation: don't use an AI-assisted editor like Cursor or Copilot yet. They'll write code for you before you understand what you're writing. Use a plain editor until you can evaluate what the AI produces.

Prerequisites

This book assumes no prior programming experience. You'll need:

- A computer running Windows, macOS, or Linux
- Python 3.9 or newer installed (instructions are provided in Chapter 2)
- A code editor (VS Code works well)
- Willingness to experiment and read error messages instead of panicking

Even if you've never written a line of code before, this book will get you there. Chapter 1 starts with what computer science actually is. Chapter 2 gets Python running.

1 The Foundations of Computational Thinking

People assume computer science is about computers the way they assume astronomy is about telescopes. The telescope is the tool, but the stars are the subject. In computer science, the computer is the tool - the subject is problems: how to define them precisely, how to solve them efficiently, and occasionally how to prove they can't be solved at all.

This distinction shapes everything that follows: you could spend a career writing code without doing much computer science, and you could do important computer science with no code at all. What defines the field isn't the medium, it's the questions: What exactly is this problem? Is there an algorithm to solve it? Is there a better one? What does it even mean being better?

Let's start with a problem.

1.1 Finding a Name in a Phone Book

You need to find "Guido" (the creator of Python) in a programmer's paper phone book with thousands of names, you don't have a search box. How do you do it?

Phone books aren't really used anymore, but the problem maps cleanly to any situation where you're searching a list without built-in search: finding a contact in an old phone, looking up a word in a paper dictionary, scanning a sorted spreadsheet.

One approach: start at the first page. Check the first name; is it "Guido"? No. Check the second. No. Keep going until you find it or run out of names.

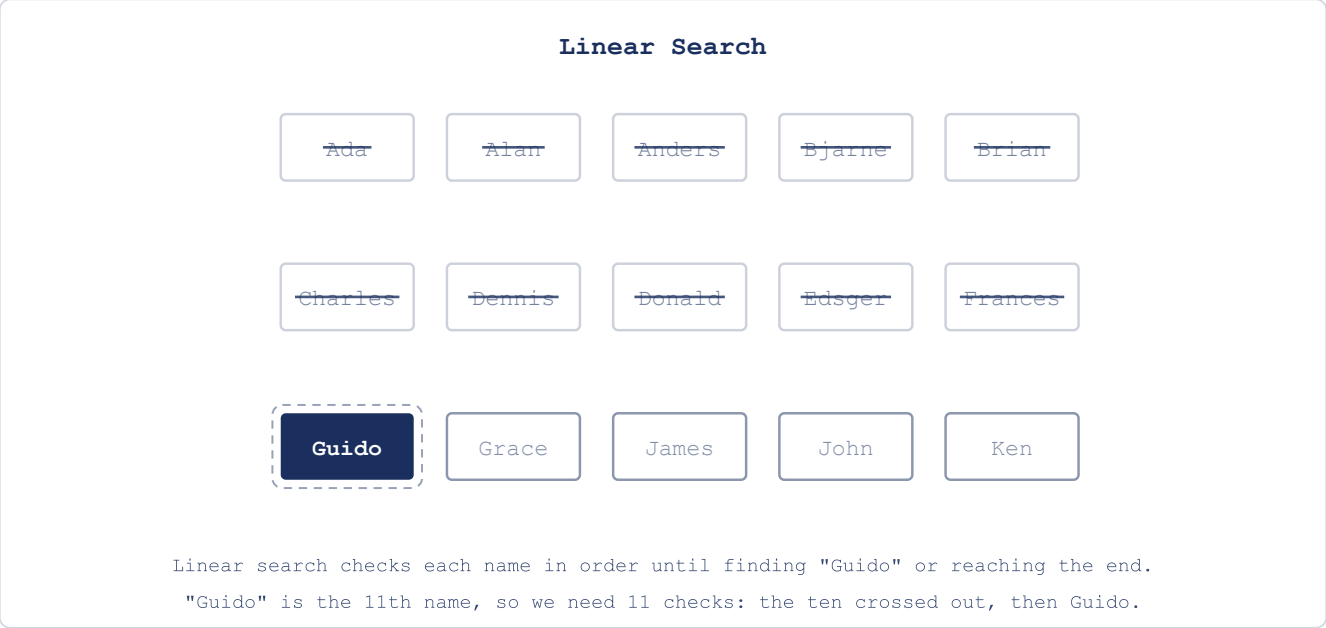


Figure 1.1: Linear Search Visualization

This works, it will find the name if it's there since you're potentially scanning every element in the phone book. But notice what happens as the phone book grows: a phone book with 1,000 names might require 1,000 checks, a million names: a million checks. If checking one name takes one second, a million-entry city phone book could take nearly two weeks assuming you get no sleep. That's a solution, but not a practical one.

Now think about how you actually use a phone book. You don't start at page one, you flip to roughly the middle. Why is that? We are unconsciously using the fact that the book is alphabetically ordered, and if we're searching for a name that starts with "M" it's probably closer to the center. Let's also use this fact in our method:

1. Open to the middle of the phone book
2. Check the name at that position
3. If it's "Guido," you're done
4. If "Guido" comes before that name alphabetically, search only the first half
5. If "Guido" comes after, search only the second half
6. Repeat with the chosen half, again splitting it in the middle
7. Continue until you find the name or determine it's not there

Let's trace this on a 20-name phone book. Open to the middle and you land on "James."

Figure 1.2: Binary Search Step 1

"Guido" comes before "James" alphabetically. Discard the second half. Ten names left. Open to the middle of those: "Donald."

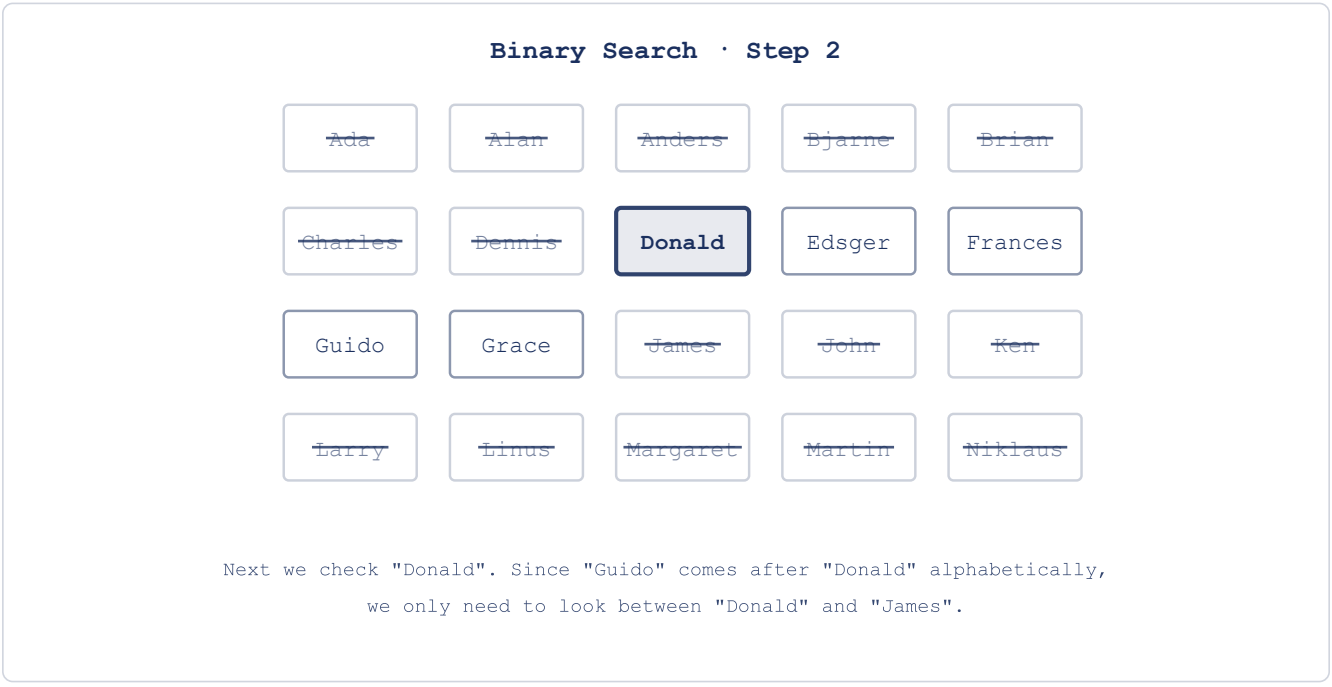


Figure 1.3: Binary Search Step 2

"Guido" comes after "Donald." Down to 5 names, all between "Donald" and "James." Middle of that section: "Frances."

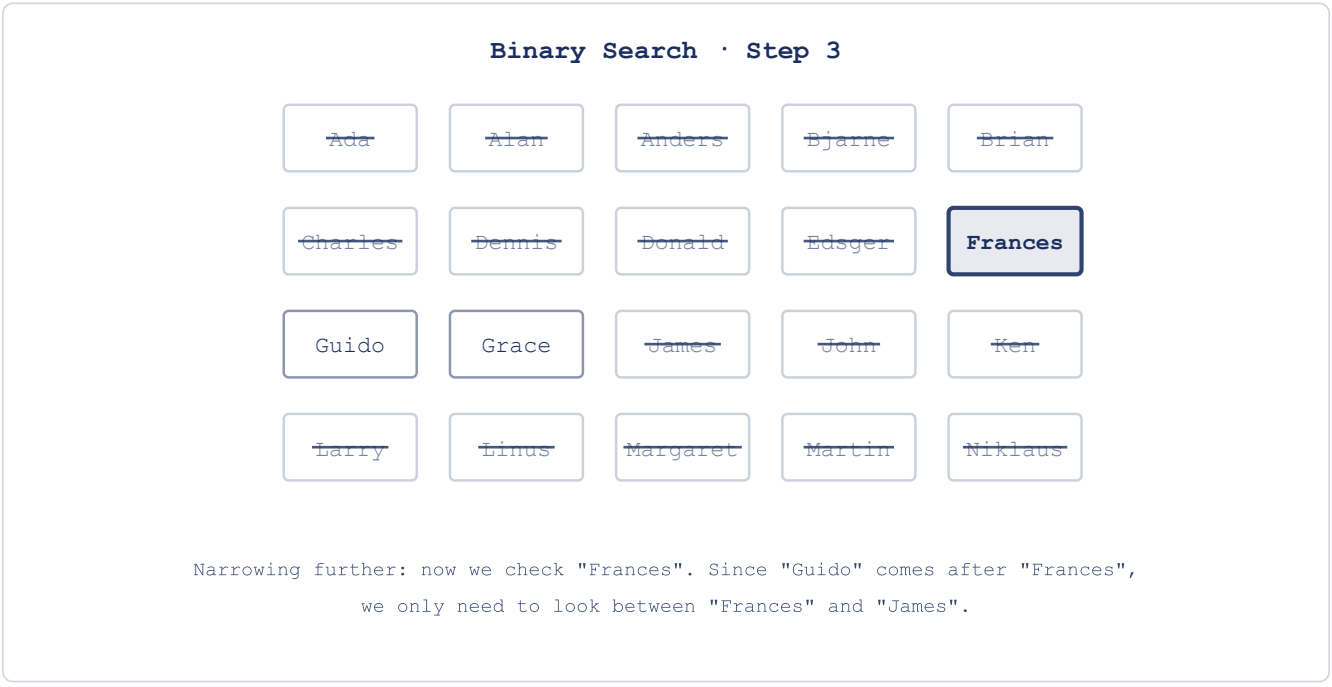


Figure 1.4: Binary Search Step 3

"Guido" comes after "Frances." Two or three names left. Check the middle: "Guido." Found it.

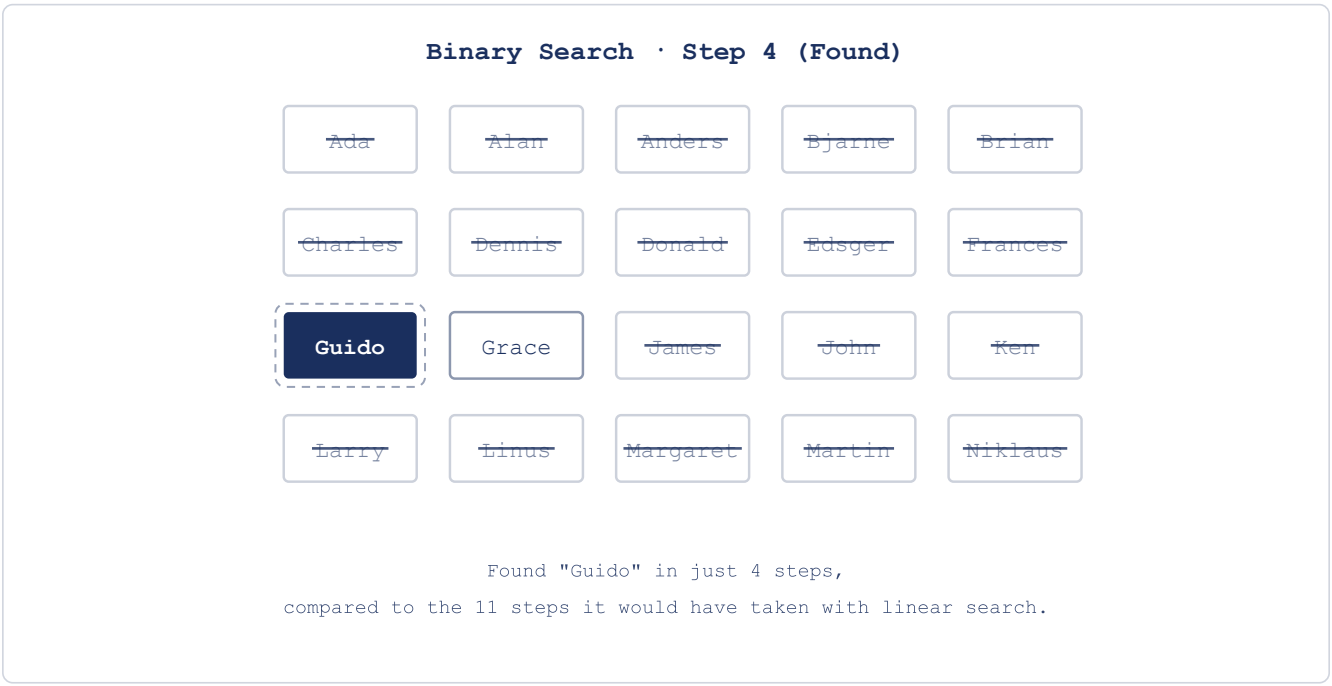


Figure 1.5: Binary Search Step 4

Four checks. The search space halved at every step: 20 names, then 10, then 5, then 2–3, then done.

Let's compare the two approaches across phone books of different sizes:

Phone Book Size	Linear Search	Binary Search
1,000 names	1,000 checks	10 checks
1,000,000 names	1,000,000 checks	20 checks
8,000,000 names	8,000,000 checks	23 checks

Eight times as many names adds millions of checks to linear search,
but only three more steps to binary search. That gap only widens with size.

Figure 1.6: Search Algorithms Comparison

For 8 million names, if each check takes one second: the first approach takes at most 8 million seconds (over three months), the second takes at most 23 seconds. Both approaches are correct, but one of them is actually usable.

Notice what "better" actually meant there: not that binary search wins on one particular phone book - it's that as the book grows, binary search's cost barely grows with it, while linear search's climbs right alongside the size. That's almost always what we mean when we call one algorithm better than another: not that it's faster on some specific input, but that it's slower to get worse as the input grows. It's a small shift in the question - from "how long does this take?" to "how does the cost grow as the problem grows?". We'll come back to it later with a language of its own. Steps aren't the only thing a solution spends, either - memory is the other one. A recurring move in everything ahead is trading one for the other: spending some memory to make an algorithm faster, or accepting more steps to use less space.

> Each time you double the phone book's size, the first approach takes twice as long, the second only needs one more step. This is what computer scientists call *logarithmic scaling*, one of the most powerful ideas in algorithm design.

We've been talking about "approaches" and "methods." Computer science has a precise word for what we've described: an **algorithm**.

1.2 What Makes an Algorithm?

Both phone book approaches share certain properties: they're sequences of steps, they take an input (a name to find, a phone book to search), they produce an output (the name's location, or the fact that it's

not there), they terminate: you always finish, whether or not the name exists.

That's what an **algorithm** is: a finite, well-defined sequence of computational steps that takes an input and produces an output. The word "finite" is important here: an algorithm must terminate. "Well-defined" is also important: each step must be unambiguous: precise enough that a machine (or a person following instructions mechanically) could execute it without guessing.

The phone book showed us two algorithms for the same problem - the first is called **linear search**: check each element in sequence, the second is called **binary search**: use the sorted structure to halve the search space at every step. Both are correct but one is dramatically better for our use-case. For small inputs, efficiency barely matters, but for large inputs, algorithm choice determines whether a solution is practical at all.

Let's think about the two questions we just asked about these algorithms. First: is it correct - does it actually produce the right answer, every time, not just on the examples we happened to try? Both of our searches pass this one. Second: how does it scale - what happens to its cost as the input grows? Here the two split apart completely. Almost everything you'll do from here on is some version of these two questions: get it right, then get it to scale.

Binary search also reveals something subtler: it works because the phone book is sorted. If names were in random order, you couldn't discard half the book with each comparison, so the structure of the data enabled a shortcut. You'll see this pattern showing up everywhere: finding structure in your problem often unlocks better algorithms. The algorithm and the way the data is arranged aren't separate concerns - they're two halves of the same thing. The right structure is often what makes the algorithm you want possible in the first place. A big part of what's ahead is really about organizing data so that the good algorithms open up to you - lists, dictionaries, sets, and the rest make better solutions possible. That structure wasn't free, though - someone sorted the book once so that every search afterward could be cheap. Paying a cost up front to make repeated work fast is a trade you'll see constantly.

But understanding an algorithm and being able to explain it precisely are two different things. You followed binary search intuitively, but could you write it down well enough for someone who has never seen a phone book?

1.3 On Precision

Consider an algorithm for making a sandwich:

1. Take two slices of bread
2. Spread mayonnaise on one slice
3. Add lettuce, tomato, and cheese
4. Place the second slice on top
5. Cut the sandwich in half

A human understands this instantly, but read it again as if you've never seen a sandwich. Which side of the bread gets the mayonnaise? How much? How should the tomato be sliced? Where does each ingredient go? What does "cut in half" mean: diagonally? Down the middle?

You fill in these gaps without even thinking; you've made sandwiches, you know what bread is, you have common sense - a computer has none of that. It needs exact quantities, exact positions, exact actions. Every unstated assumption, every common-sense inference, has to be made explicit.

Go back to binary search. "Open to the middle of the phone book" is clear enough for you. But a computer needs to know: how do you compute "the middle" of a list? If the list has an even number of entries, which of the two middle entries do you check? When you "discard the second half," do you include the entry you checked, or not? These details don't change whether the algorithm works in principle, but they're the difference between a rough spec and an actual implementation, you'll need to deal with these things.

This is what makes programming difficult when you're starting out, you're shifting from implicit to explicit: you have to say what you mean completely, not rely on the reader to fill in the rest. A machine that follows instructions exactly (with no assumptions, no improvisation) can do it billions of times without fatigue. But it will only do exactly what you said, not what you meant.

Our search problem had a really simple scope, but what happens when the problem is bigger than a single search?

1.4 Breaking Problems Apart

Finding a name in a phone book is one task, building a contacts app involves many: adding new contacts, sorting them, searching by name, searching by number, handling duplicates, displaying results. Each of these is its own problem with its own algorithm. the skill is recognizing where a large problem divides into smaller, independent pieces.

This is called **decomposition**: breaking a complex problem into subproblems, each manageable on its own. It mirrors what binary search does to the phone book - split the problem in half, solve the smaller version, a strategy common enough to have a name: divide and conquer - but applied to the problem's structure rather than its data.

Decomposition alone isn't enough, though. Once you've broken a problem into pieces, you need a way to work on each piece without drowning in the details of all the others.

1.5 Abstraction

When you described binary search to a friend, you probably said something like "open to the middle, check the name, throw away half." You didn't explain how pages work or how alphabetical ordering is defined. You worked at a level where those details didn't really matter to understand the whole.

This is **abstraction**: hiding what you don't need to think about so you can focus on what you do.

When you use a smartphone, you don't think about transistors or voltage levels or memory addresses, you simply tap an icon. The icon is an abstraction: an interface that hides enormous complexity underneath. You don't need to understand what's happening at the hardware level to use the app, the abstraction lets you work at a higher level.

Programming languages work the same way. When you call `print()` in Python, you're not writing the machine instructions that move characters to a screen buffer, you're using an abstraction that handles all of that for you. Python itself is an abstraction over assembly code, which is an abstraction over machine code, which is an abstraction over electrical signals running through the processor.

Binary search has layers too. At the top: "halve the search space until you find the target". One level down: "compare the target to the middle element and decide which half to keep". One level further: "compute the index of the middle element given the current bounds". You can think about the strategy without thinking about index arithmetic, and you can think about index arithmetic without thinking about how memory stores the list. Each layer trusts the one below it.

This layering is how complex systems can get built at all: you can work at one level without understanding (or even having to think about) the levels below. When you need to go deeper, you can, but most of the time, the abstraction holds and you don't have to.

We've been describing algorithms in words and diagrams, but at some point they need to run on a computer. That requires translating them into a language a computer can execute.

1.6 From Algorithms to Programs

An **algorithm** is a logical sequence of steps to solve a problem: language-independent, the idea itself; a **program** is that algorithm implemented in a specific programming language: one particular expression of the idea.

Binary search is an algorithm, and you could implement it in Python, Java, C++, or any other language. The algorithm stays the same; the syntax changes. Programming is the act of transcribing an algorithm into a language a computer can execute.

This wasn't always done in high-level languages though, early programmers gave instructions directly in machine code: the ones and zeros the processor actually understands. A program to display "Hello, World!" in machine code looks like this:

```
B4 03 CD 10 B0 01 B3 0A B9 0D 00 BD 13 01 B4 13 CD
10 C3 48 65 6C 6C 6F 2C 20 57 6F 72 6C 64 21 0D 0A
```

Broken down, those hexadecimal codes represent specific processor instructions (This is real-mode DOS assembly, from back when programmers worked this close to the metal - it's here to look at, not to run. It won't execute on a modern 64-bit machine as-is):

```
B4 03       ; MOV AH, 03h     (Get cursor position)
CD 10       ; INT 10h         (BIOS video interrupt)
B0 01       ; MOV AL, 01h     (Write mode parameter)
B3 0A       ; MOV BL, 0Ah     (Color attribute)
B9 0D 00    ; MOV CX, 000Dh   (Character count - 13 for "Hello, World!")
BD 13 01    ; MOV BP, 0113h   (Offset to string data)
B4 13       ; MOV AH, 13h     (Write string function)
CD 10       ; INT 10h         (BIOS video interrupt)
C3          ; RET             (Return)
```

And the text data itself, encoded as numbers:

```
48 65 6C 6C 6F 2C 20 57 6F 72 6C 64 21 0D 0A
```

You don't need to understand this, you need to feel the gap between that and the Python equivalent:

```python
print("Hello, World!")
```

Same result, one line versus dozens of cryptic instructions. That gap is what high-level programming languages exist to close (and it's abstraction at work: `print()` hides all of that machine-level complexity behind a single word).

Here's another example, this algorithm finds the maximum value in a list:

1. Start with the first number as the current maximum
2. For each remaining number, if it's larger than the current maximum, it becomes the new maximum
3. After checking all numbers, the current maximum is the answer

And here's what it looks like in Python (don't worry about the syntax for now):

```python
def find_maximum(numbers):
    if not numbers:
        return None

    maximum = numbers[0]

    for number in numbers[1:]:
        if number > maximum:
            maximum = number

    return maximum

my_numbers = [42, 17, 8, 94, 23, 61]
print(f"The maximum value is {find_maximum(my_numbers)}")
```

The algorithm is the logic, the Python code is one way to express it. Keeping these separate in your head is important because when your program gives the wrong answer, the problem is either in the algorithm

(the logic is wrong) or in the implementation (the code doesn't correctly express the logic) - different bugs with different fixes.

The choice of which language to use is a separate question, and for this book, it's already been made for you.

1.7 Why Python?

Python was created by Guido van Rossum in the late 1980s. Its central design philosophy, summarized in a document called "The Zen of Python," includes lines like:

- Beautiful is better than ugly
- Explicit is better than implicit
- Simple is better than complex
- Readability counts

These rules make Python code read a lot like pseudocode: instructions close enough to plain English that the algorithm is visible in the implementation. For learning, that's valuable; you can focus on the problem without fighting the language.

Python is also practical since it's the dominant language in data science, machine learning, scientific computing, and automation. Learning it doesn't only teach you to program; it gives you access to a vast ecosystem of tools built by other people solving related problems.

It has real limitations though, and you should know them to be able to decide when it's the best option. Python is interpreted: instead of being compiled all the way to machine code ahead of time, your code is translated into an intermediate form (called bytecode) and executed step by step as it runs. That extra work at runtime is a big part of why compiled languages like C++ or Rust are typically much faster. For most applications this doesn't matter, but for performance-critical or low-resource systems, it does. Python also has a mechanism called the Global Interpreter Lock (the **GIL**) that allows only one thread to execute Python code at a time, which limits certain kinds of parallel processing. Note that the GIL can be disabled since python 3.13, but it will take some time until packages adapt and fix the bugs that the GIL was previously suppressing. And some domains (mobile apps, game engines, low-level systems programming) are better served by other languages. Every language is a tradeoff, python is optimized for programmer productivity and the scientific and data ecosystem.

Professional programmers typically know several languages and choose based on the problem. The concepts you'll learn here (algorithms, data structures, abstraction) apply in any language, python is a clear place to start.

We know that the concepts matter more than the tool, but this raises one more question: what kinds of problems can these concepts actually solve?

1.8 What Makes a Problem Computational?

Not every problem can be solved with an algorithm. We've been working with "find a name in a phone book," which is clearly solvable: there's a finite list, a defined target, and a procedure that terminates, but consider "what is the most beautiful painting?" - there's no unambiguous definition of "beautiful," no finite set of candidates everyone agrees on, and no procedure that would produce an answer everyone accepts. It's not a computational problem.

For a problem to be computational, it needs to be well-defined (the problem statement is clear and unambiguous), solvable (there exists a finite sequence of steps that reaches an answer), and finite (the solution must terminate). "What is the sum of all integers from 1 to 100?" is computational. "What career would make someone happiest?" is not.

Computational thinking is the practice of reformulating problems so they become solvable algorithmically. This often involves ignoring irrelevant details (abstraction), breaking the problem into pieces (decomposition), and recognizing when a new problem resembles one you've already solved (pattern recognition). It's the collection of concepts we've been defining throughout this chapter, applied deliberately.

The phone book showed us all of this in miniature: we started with a problem, we found two algorithms and discovered that the choice between them matters enormously, then we saw that the data's structure (sorted order) enabled the better algorithm. We also used decomposition and abstraction to manage complexity, then distinguished the algorithm (the idea) from the program (its implementation in a specific language).

These are the ideas the rest of the book applies. In the next chapter, we move from theory to practice: setting up Python and writing actual programs, but always starting with the thinking.

1.9 Chapter Summary

Computer science is primarily about solving problems, not about computers. The phone book made the stakes concrete: linear search and binary search both find the name, but at scale the difference between them is the difference between seconds and months; algorithm choice dominates.

An algorithm is a finite, well-defined sequence of steps that takes an input and produces an output. "Finite" and "well-defined" are both essential. Binary search also showed that the structure of the data (sorted order) enabled the better algorithm. Finding structure in your problem is often what unlocks a better solution.

The sandwich example exposed the precision gap: humans tolerate vague instructions, computers don't. Programming requires you to close that gap, stating every step explicitly enough that a machine can follow without guessing. Decomposition breaks large problems into manageable pieces. Abstraction lets you work at one level without drowning in the details of the levels below. Both are how complex systems get built at all.

Algorithms are language-independent descriptions of solutions. Programs are their implementation in a specific language. Keeping those two things distinct makes debugging much cleaner: when something goes wrong, is the logic flawed or did the code fail to express the logic correctly?

In the next chapter, we set up Python and write actual programs. The exercises below are worth doing even if they seem obvious; they're not testing what you know, but wiring the circuits.

1.10 Exercises

1. Write a detailed algorithm for a common daily task: making breakfast, getting ready in the morning, or anything you do regularly. Be precise enough that someone who has never done it could follow your instructions without guessing. Once you're done, go back and ask: where did you make assumptions? What did you leave implicit that a computer would need stated explicitly? The gap between your first draft and a truly unambiguous version is the gap that programming requires you to close.

2. Using the sorted list of 16 programming languages below, pick any language other than Python and trace both algorithms to find it. For linear search, check each language in order and count the comparisons; For binary search, use divide-and-conquer and count the comparisons, then write out every step of both. The difference in step counts is the point, but try to also articulate *why* binary search needs so many fewer steps.

```
1. Ada        5. Cobol     9. JavaScript  13. Python
2. C          6. Fortran  10. Kotlin       14. Ruby
3. C#         7. Go       11. Perl         15. Rust
4. C++        8. Java     12. PHP          16. Swift
```

3. Choose one of these complex tasks and break it into subtasks, each precise enough to be its own algorithm: planning a vacation, organizing a birthday party, or writing a research paper. The goal is identifying the natural seams in the problem, the places where a complex task divides into independent pieces. How deep do you need to go before each piece feels manageable?

4. For each of the following, decide whether it's computational (solvable by algorithm) or not, and explain your reasoning: (a) finding the shortest route between two cities, (b) determining the most beautiful musical composition, (c) calculating the average temperature for a city over a year, (d) converting currencies based on current exchange rates, (e) deciding which career would make someone happiest. Some of these are less obvious than they first appear.

5. Below is a 28-step grocery shopping trip written out in full. Group the steps into meaningful functions and rewrite the whole trip using only those function names, so the high-level description fits in a few lines; the complexity lives inside the named abstractions, not at the top level. Then ask yourself: does the abstraction actually hide the right things? Are there groupings that feel arbitrary, and why?

```
1. Write down needed items
2. Check what you already have at home
3. Update shopping list
4. Get reusable bags
5. Find car keys
6. Drive to grocery store
7. Park the car
8. Grab a shopping cart
9. Enter the store
10. Check first item on list
11. Find the correct aisle
12. Locate the item
13. Check price and quality
14. Place item in cart
15. Cross item off list
16. Repeat steps 10-15 until list is complete
17. Proceed to checkout
18. Wait in line
19. Place items on conveyor belt
20. Pay for groceries
21. Bag the groceries
22. Return cart
23. Walk to car
24. Load bags into car
25. Drive home
26. Carry bags inside
27. Unpack groceries
28. Store items in proper places
```

6. The binary search algorithm in this chapter relies on the phone book being sorted. What would happen if it weren't? Try tracing binary search on an unsorted list of 8 names (make one up). At what point does the algorithm break down? What does this tell you about the relationship between an algorithm and the assumptions it makes about its input?

2 Getting Started with Python

Hopefully you understand binary search and could explain it to someone now, but explaining it to a computer requires a level of precision that English doesn't demand: exact syntax, explicit instructions, no unstated assumptions. In this chapter we'll move from the abstract specification to the concrete implementation.

We'll set up Python, learn how to run code, and build a small program that takes input from a user, processes it, and produces formatted output, which is the structure of most programs you'll ever write - an input-process-output loop:

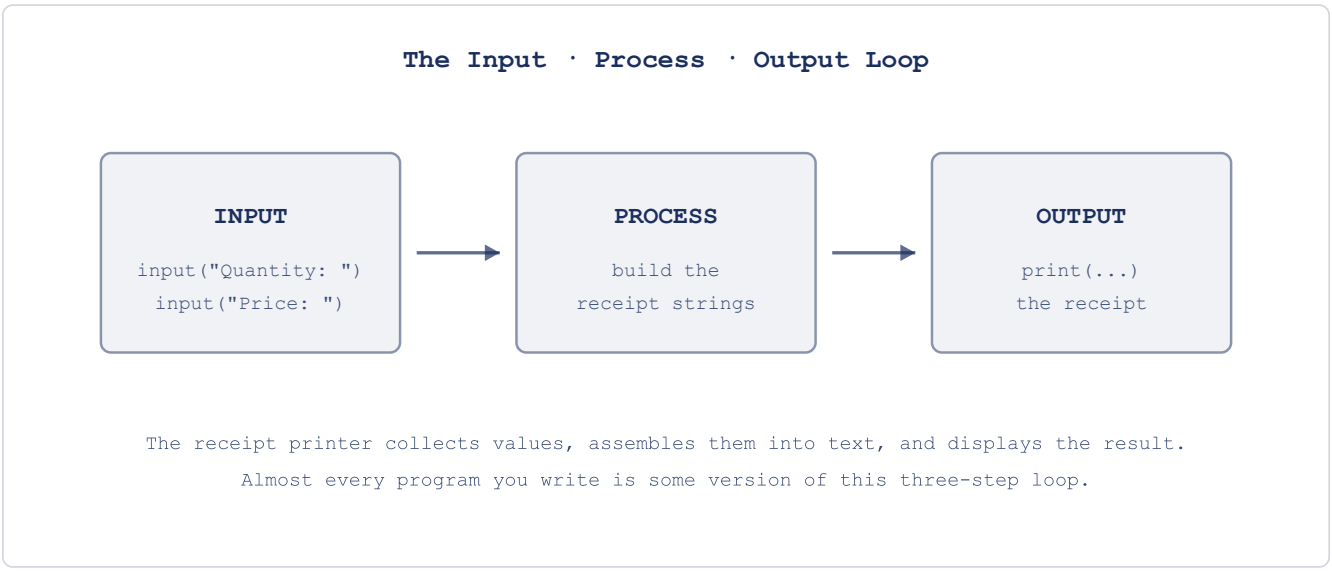


Figure 2.1: The Input-Process-Output Loop

By the end of the chapter you'll have written one.

2.1 Installing Python

We'll use a single tool for everything: **uv**. It installs Python for you, manages your projects, and keeps each project's dependencies isolated - jobs that traditionally took three or four separate tools. Learning one thing now saves you a lot of confusion later.

Install `uv` first. It runs on Windows, macOS, and Linux.

On **Windows**, open PowerShell and run:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

On **macOS or Linux**, open a terminal and run:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Close and reopen your terminal so it picks up the new command, then check it worked:

```
uv --version
```

If you see a version number, you're set. Notice what you *didn't* have to do: you never installed Python directly. `uv` does that on demand - the first time a project needs a particular Python version, it fetches it for you. You can also do it explicitly:

```
uv python install
```

For a code editor, Visual Studio Code is the right choice for this book. It's free, runs on all platforms, and has solid Python support through its Python extension. Download it from code.visualstudio.com and install the Python extension from the Extensions panel. If you can't install software locally, pydantic.run is an online Python environment that works in a browser with no setup at all - a good fallback for getting started, though you'll want a local setup for the later chapters.

One recommendation: don't reach for an AI-assisted editor like Cursor or Copilot yet. You'll use these tools eventually - they're genuinely useful - but right now they'll write the code for you before you understand what you're writing, which makes the early chapters pointless. The struggle is a big part of how the ideas actually stick. Later, the right use of these tools is to help you write code faster, not to think for you - and this book is about learning to think. Use a plain editor until you can judge for yourself whether what the AI produces is any good.

Once `uv` is installed, there are two main ways to run Python code.

2.2 Two Ways to Run Code

The first way is interactive. Open a terminal, run `uv run python`, and you'll see a prompt that looks like `>>>`. This is the **REPL** (Read-Evaluate-Print Loop): you type an expression, Python evaluates it, prints the result, and waits for the next input. (`uv run` just makes sure you're using the Python that `uv` manages, rather than whatever else might happen to be on your system.)

```
>>> 2 + 3
5
>>> "Hello" + " " + "World"
```

```
'Hello World'
```

The REPL is useful for testing small ideas quickly: does this expression do what I think? You'll use it throughout the book to check your understanding before committing something to a file. Type `exit()` or press Ctrl+D (Ctrl+Z on Windows) to leave it.

The second way is script mode, you write code in a file with a `.py` extension and run the whole file at once. Create a file called `first.py` with this content:

```
2 + 3
"Hello" + " " + "World"
```

Run it with `uv run first.py`. Nothing appears. That's not a bug: in script mode, Python evaluates expressions but doesn't print results unless you explicitly ask it to. This is the first taste of the precision gap from Chapter 1: you probably assumed Python would show you the result. Python did exactly what you said, which was "compute this value", you never said "show it to me."

That's what `print()` is for.

2.3 Telling the Computer What to Show

Fix the script:

```
print(2 + 3)
print("Hello" + " " + "World")
```

Now it outputs:

```
5
Hello World
```

`print()` takes whatever you give it and displays it on screen.

You can print multiple values in a single call by separating them with commas and Python inserts a space between them:

```
print("The answer is", 42)
```

```
The answer is 42
```

By default `print()` adds a newline after each call, you can change that with the `end` parameter: `print("Hello", end=" ")` leaves the cursor on the same line and just adds a space so the next `print()` continues there. The default value of `end` is `"\n"`, which Python interprets as a new line.

Now we can display things, but everything so far is hardcoded: the program does the same thing every time. To build anything useful, we need to work with values that change, so we need the computer to remember things.

2.4 Remembering Values

A **variable** is a name that refers to a value. You create one with the assignment operator =:

```
message = "Hello, Python!"
number = 42
print(message)
print(number)
```

```
Hello, Python!
42
```

The left side is the name, the right side is the value. Once assigned, the name can be used anywhere you'd use the value directly. This is how programs hold onto information: give it a name that references where the value is in the memory, then use that name later.

Variables can be reassigned:

```
count = 10
count = count + 1
print(count)
```

```
11
```

The second line reads the current value of `count`, adds 1, and assigns the result back to `count`. This pattern is constant in programming: it's how counters, accumulators, and running totals work.

Variable names can contain letters, numbers, and underscores, but can't start with a number and can't be Python keywords like `if` or `for`. Python convention is lowercase words separated by underscores: `user_name`, `total_score`, `highest_temperature`. Names are case-sensitive: `score` and `Score` are different variables. Also, use descriptive names (`user_age` instead of `x`); you'll thank yourself later.

Now we can display things and remember them, but the program still does the same thing every time it runs, because every value is written into the code. A useful program responds to whoever is using it, it needs to ask questions.

2.5 Asking the User

`input()` pauses the program, displays a prompt, waits for the user to type something and press Enter, then returns what they typed as a string (how Python stores text).

```
name = input("What is your name? ")
print("Hello, " + name + "!")
```

When this runs, the user sees `What is your name?` and can type a response; whatever they type gets stored in `name` and used in the greeting. This is the first program we've written that behaves differently depending on who runs it.

One thing to keep in mind: `input()` always returns a string, even if the user types a number. If you ask someone for their age and they type `25`, you get the string (that is, the text) `"25"`, not the integer number `25`. This distinction between text and numbers will be important very soon.

We now have four tools: `print()` to display output, expressions to compute values, variables to remember them, and `input()` to get information from the user. Before we combine them, we need to understand how Python handles the two main kinds of data we'll work with: numbers and text.

2.6 Numbers and Text

Python supports the arithmetic operators you'd expect: `+` for addition, `-` for subtraction, `*` for multiplication, `/` for division. Division always returns a decimal number (one with a whole number part and a fractional part, separated by a decimal point), even when the result is a whole number: `10 / 2` gives `5.0`, not `5`. Two additional operators are worth knowing early: `**` for exponentiation (`2 ** 8` gives `256`) and `%` for the remainder after division (`10 % 3` gives `1`).

The order of operations follows standard math: exponentiation first, then multiplication and division, then addition and subtraction; parentheses override the default order: `2 + 3 * 4` gives `14`, but `(2 + 3) * 4` gives `20` (PEMDAS).

Text in Python is called a **string**, you write strings in either single (`'`) or double (`"`) quotes; the choice doesn't matter as long as you're consistent within a single string. Strings support two operators that look familiar but behave differently. `+` concatenates them: `"Hello" + " " + "World"` gives `"Hello World"`. `*` repeats them: `"ha" * 3` gives `"hahaha"`.

Notice the difference: when Python sees `2 + 3`, it adds the numbers and gets `5`. When it sees `"Hello" + " World"`, it glues the strings together and gets `"Hello World"`. The same operator behaves differently depending on the data type, and this is important: `"2" + "3"` gives `"23"`, not `5`. The quotes make them strings, so `+` concatenates instead of adding.

Remember that `input()` always returns a string? If the user types `5` and you store it in a variable called `price`, you have the string `"5"`, not the number `5`. You can concatenate it with other strings (`"Price: $" + price` gives `"Price: $5"`), but you can't do math with it, at least not yet. We'll fix this in Chapter 3 when we introduce type conversion.

That's the full toolkit for this chapter: output, expressions, variables, input, and the beginnings of understanding data types. Time to use them together.

2.7 Hands-On: Personal Receipt Printer

We'll build a program that collects information about a purchase, formats it, and prints a receipt. It uses every concept from the chapter and produces something you can look at and modify. It also runs straight into a wall that will make the need for Chapter 3 concrete.

One new piece of syntax shows up in the code below: a line that starts with `#` is a **comment**. Python ignores it completely - it's not an instruction, it's a note to yourself (or to whoever reads the code later). We'll use them to label things and leave reminders. Anything after the `#` on that line is invisible to Python.

Set up your project:

```
uv init chapter2
cd chapter2
```

`uv init` creates a small project scaffold - a folder with a couple of config files `uv` uses to track your project. You don't need to worry about what's in them yet; we'll look properly in the modules chapter, near the end of the book. Create a file called `receipt.py` inside this folder. We'll build it up in pieces.

Start by collecting the input:

```
# receipt.py
print("=== Receipt Printer ===", end="\n\n")

store_name = input("Store name: ")
item_name = input("Item name: ")
quantity = input("Quantity: ")
price = input("Price per item: ")
customer_name = input("Customer name: ")
```

If you run it with `uv run receipt.py`, you should be prompted for each piece of information in order. The program collects it but doesn't do anything with it yet.

Now add the receipt output below the input section:

```
# receipt.py
# ...existing code above

print()
print("=" * 30)
print("           " + store_name)
print("=" * 30)
print()
print("Customer: " + customer_name)
```

```
print()
print(item_name)
print("  " + quantity + " x $" + price)
print()
print("-" * 30)
print("  Total items: " + quantity)
print("=" * 30)
print("    Thank you for your purchase!")
print("=" * 30)
```

Run it again and enter some values. If you enter "Bookshop" as the store, "Book" as the item, "2" as the quantity, "14.99" as the price, and "Hannah" as the customer, the receipt looks like:

```
==============================
           Bookshop
==============================

Customer: Hannah

Book
  2 x $14.99

------------------------------
  Total items: 2
==============================
    Thank you for your purchase!
==============================
```

Notice that `"=" * 30` produces a row of thirty equals signs. The `*` operator on strings repeats the string, which is a common way to draw lines in terminal output.

Now try to add a total. You have the quantity and the price, the total should be quantity times price. Try adding this line:

```
print("  Total: $" + quantity * price)
```

Run it and you'll see that Python crashes. The error says something about not being able to multiply a string by a string. `quantity` and `price` are strings (because `input()` always returns strings), and Python doesn't know how to multiply `"2"` by `"14.99"`. You can't compute `"2" * "14.99"` any more than you can compute `"hello" * "world"`, the operation doesn't make sense for text.

The receipt captures the pattern (collect input, build strings, format output), but it can't do the one thing a real receipt needs, which is calculating the total. We're treating numbers as text, and text can't do arithmetic. Chapter 3 fixes this by introducing data types properly, once you can write `float(price) * int(quantity)`, the receipt works for real.

Remove the broken total line for now. The receipt is complete as a string-formatting exercise. Try modifying it: change the formatting, add a date line, widen the receipt; breaking things and seeing what happens is how you actually learn what the rules are.

2.8 Chapter Summary

Setting up Python is overhead, but it only happens from time to time. With `uv` you install one tool and it takes care of fetching Python, creating projects, and keeping them isolated. The two modes of running code (interactive and script) serve different purposes: the REPL for quick experiments, script files for anything you want to keep. The first surprise was that script mode doesn't show results unless you ask: Python does what you say, not what you assume.

`print()` is how programs communicate output; `input()` is how they accept it. Variables store values so they can be reused and combined. These are the minimum tools for the input-process-output loop that most programs follow. The receipt printer demonstrated that, but it also demonstrated a wall: `input()` returns strings, and strings can't do math. The program can display prices but can't add them up, can show quantities but can't multiply them.

That wall is the data type system. In the next chapter, we'll introduce types properly: integers, floats, strings, and the conversions between them.

2.9 Exercises

1. Extend the receipt printer to support three line items instead of one. Ask for a name, quantity, and price for each item separately. Format them as three rows in the receipt body. You'll have three `item_name`, `quantity`, and `price` variables (name them clearly: `item1_name`, `item2_name`, etc.). The total line still can't be calculated yet: display the individual quantities. Which part of the output is hardest to format correctly, and why?
2. Write a program that asks for a sender's name, a recipient's name, and a one-sentence message, then formats and prints a postcard. The postcard should have a border made of a repeated character, the recipient prominently at the top, the message in the middle, and "From: [sender]" at the bottom. The border width should be determined by the length of the message (`"=" * len(message)` gives you a line exactly as wide as the message). What happens if the message is very short or very long?
3. Write a program that asks for someone's first name, last name, and preferred title (Mr., Ms., Dr., etc.), then prints their name formatted three ways: full formal (`Dr. Jane Smith`), last name first (`Smith, Jane`), and initials (`J.S.`). The initials require you to access individual characters in a string: `name[0]` gives you the first character. What happens if you use `name[0]` on an empty string?

4. The `print()` function accepts a `sep` parameter that controls what gets placed between multiple values instead of the default space: `print("a", "b", "c", sep="-")` prints `a-b-c`. Write a program that asks for five words and prints them in three formats: space-separated (default), comma-separated, and with a pipe character between them (`word1|word2|word3`). Now try passing all five variables to a single `print()` call with the appropriate `sep`. How does that compare to building the string manually with +?

5. Write a program that generates a name tag for a conference badge. Ask for the person's name, their job title, and the company they work for. The badge should have a fixed width of 30 characters, with each line of text centered. Python strings have a `.center(width)` method: `"hello".center(30)` pads the string with spaces so it's centered in a field of 30 characters. Use this to format the badge. What happens when a name is longer than 30 characters?

6. Write a program that asks for two words and prints every combination of them with a separator between them: the first word, then the second, then both joined, then the second first and the first second, then the first repeated twice, then the second repeated twice. Six lines of output from two inputs. Before writing any code, write out what the six lines should look like for inputs `"black"` and `"bird"`. Does the output match what you expected? The goal isn't to memorize string operations; it's to notice whether your mental model of what + and * do on strings matches what Python actually does.

3 Types, Conversions, and Formatting

Chapter 2 ended with a crash: the receipt printer collected a quantity and a price from the user, but when you tried to compute the total with `quantity * price`, Python refused - the error said something about not being able to multiply strings. Both values looked like numbers, but `input()` returned them as text, and text can't do arithmetic.

This chapter explains why that happened and fixes it. Along the way, we'll learn how Python distinguishes between different kinds of data, how to convert between them, and how to format output so it looks the way you want. By the end, you'll rebuild the receipt printer with a working total, real currency formatting, and none of the limitations that stopped you last time.

3.1 Why the Receipt Crashed

Let's look at what Python actually told you. When you added the total line and ran the program, you got something like this:

```
Traceback (most recent call last):
  File "receipt.py", line 25, in <module>
    print("  Total: $" + quantity * price)
                         ~~~~~~~~~^~~~~~~
TypeError: can't multiply sequence by non-int of type 'str'
```

This is a **traceback**: Python's report of where and why your program died. Read it bottom-up; the last line is the diagnosis, the lines above it show the location, and the little `~~~^~~~` markers point at the exact expression that failed. `TypeError` means an operation was applied to the wrong kind of data. "Sequence" is Python's family name for ordered collections of things - a string is a sequence of characters. And "non-int of type 'str'" means the thing you tried to multiply by wasn't an integer, it was another string.

Decoded: Python knows how to multiply a string by an integer (that's the repetition you used for `"=" * 30`), but multiplying a string by a string has no defined meaning. You expected `quantity * price` to multiply 2 by 14.99, but `input()` always returns strings, so `quantity` held `"2"` and `price` held `"14.99"` - and `"2" * "14.99"` makes no more sense than `"hello" * "world"`.

This isn't a bug in Python, Python treats `"2"` and `2` as fundamentally different things: one is text (a sequence of characters that happens to look like a number), the other is a number (a value you can do

arithmetic with). They're stored differently, they support different operations, and Python won't silently guess which one you meant.

The category of data a value belongs to is called its **type**, and understanding types is how you predict what Python will do with your data - and why it sometimes refuses. Learning to read tracebacks like the one above is part of the same skill; you'll see many more of them, and the last line almost always tells you what went wrong.

3.2 Python's Core Types

If you open the REPL (`uv run python`) and try this, you'll get these results:

```
>>> type(42)
<class 'int'>
>>> type(3.14)
<class 'float'>
>>> type("hello")
<class 'str'>
>>> type(True)
<class 'bool'>
```

The `type()` function tells you what kind of data you're working with. Python has four types you'll use constantly:

An **integer** (`int`) is a whole number: 42, -7, 0, 1000000; no decimal point. Python handles integers of any size; you'll never overflow (try to store a value larger than that chunk of memory can hold), no matter how large the number gets.

A **float** is a number with a decimal point: 3.14, -0.001, 1.0. The name comes from "floating-point," a reference to how the decimal point can appear at different positions. Even `1.0` is a float, not an integer - having the decimal point is what makes the difference.

A **string** (`str`) is the text type you met in Chapter 2: any sequence of characters enclosed in quotes. `"42"` is a string, `42` is an integer, and the quotes are what visibly separate them.

A **boolean** (`bool`) has only two possible values: `True` and `False`. You'll use these heavily once we introduce conditionals in Chapter 4; for now, know they exist.

There's also `None`, a special value that represents "nothing" or "no value" - Python's way of saying a variable exists but doesn't have meaningful data yet.

The type of a value determines what you can do with it, and the same operator can behave completely differently depending on the types involved.

3.3 The Same Operator, Different Behavior

You noticed this at the end of Chapter 2, when `"2" + "3"` gave `"23"` instead of `5`. Back then it was just an observation; now we can name the mechanism: the same symbol represents completely different operations, and Python decides which one by looking at the types of the values on either side. With numbers, + adds: `5 + 3` gives `8`; with strings, it concatenates: `"5" + "3"` gives `"53"`.

This extends to `*`: with two numbers, it multiplies: `4 * 3` gives `12`; with a string and an integer, it repeats: `"ha" * 3` gives `"hahaha"`. But with two strings? Python doesn't know what `"ha" * "3"` should mean, so it raises `TypeError`.

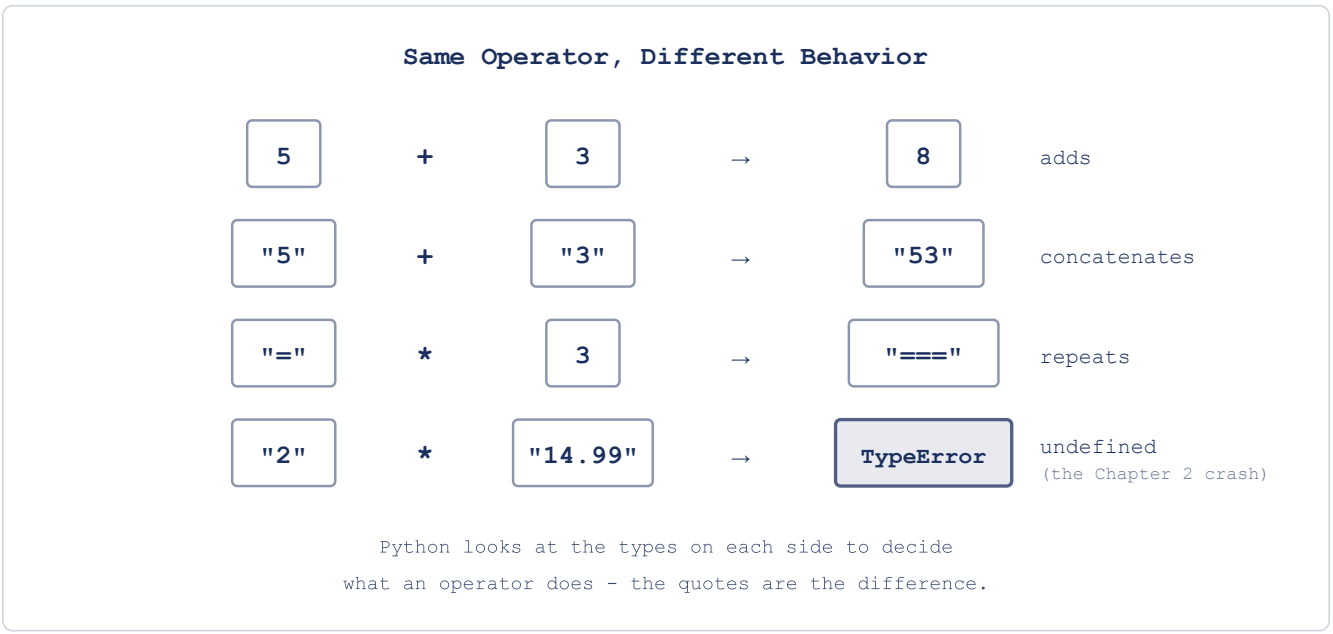


Figure 3.1: Same Operator, Different Behavior

When you mix integers and floats, Python promotes the result to a float: `5 + 3.0` gives `8.0`, not `8`. This makes sense - a float can hold fractional values an integer can't, so Python picks the more general type and nothing gets lost.

The rest of the arithmetic from Chapter 2 - subtraction, division, `**`, `%`, and the precedence rules - works exactly as you saw it there, just with these type rules layered on top. One operator we skipped is worth adding now: `//` is integer division, which divides and throws away the remainder, so `10 // 3` gives `3` while `10 % 3` gives the leftover `1` - together they split a division into its whole-number part and its remainder.

That covers how operators respond to types. But floats themselves are hiding one more surprise - and since the receipt is about to do money math with them, it's worth seeing it now.

3.4 Why 0.1 + 0.2 Isn't 0.3

Try this in the REPL:

```
>>> 0.1 + 0.2
0.30000000000000004
```

That trailing 4 isn't a Python bug, and it isn't random - every mainstream programming language gives the same answer. To see why, we have to peek one level below the abstraction, at how the computer actually stores numbers.

Everything in a computer ultimately lives in binary. Remember the abstraction ladder from Chapter 1 that ended at electrical signals running through the processor? Those signals have two states, so every number the machine holds is written using only the digits 0 and 1. Whole numbers survive the translation perfectly - 6 becomes 110, 42 becomes 101010, nothing lost. Fractions are where it gets interesting.

Some fractions can't be written exactly in a given base, and you already know one: 1/3 in our everyday base 10 is 0.3333…, the threes never end, and wherever you stop writing you've captured slightly less than 1/3. Base 2 has the same problem, just with different fractions - and one tenth is one of them.

To see why, look at what binary digits after the point even are. In base 10 the places mean 1/10, 1/100, 1/1000; in base 2 they mean 1/2, 1/4, 1/8, 1/16 - each place worth half the one before. Writing 0.1 in binary means building one tenth as a sum of these pieces. 1/2, 1/4, and 1/8 are all too big; 1/16 fits, add 1/32 and you're at 0.09375; the next pieces that fit are 1/256 and 1/512, bringing you to 0.099609375 - closer, but not there. And it never gets there:

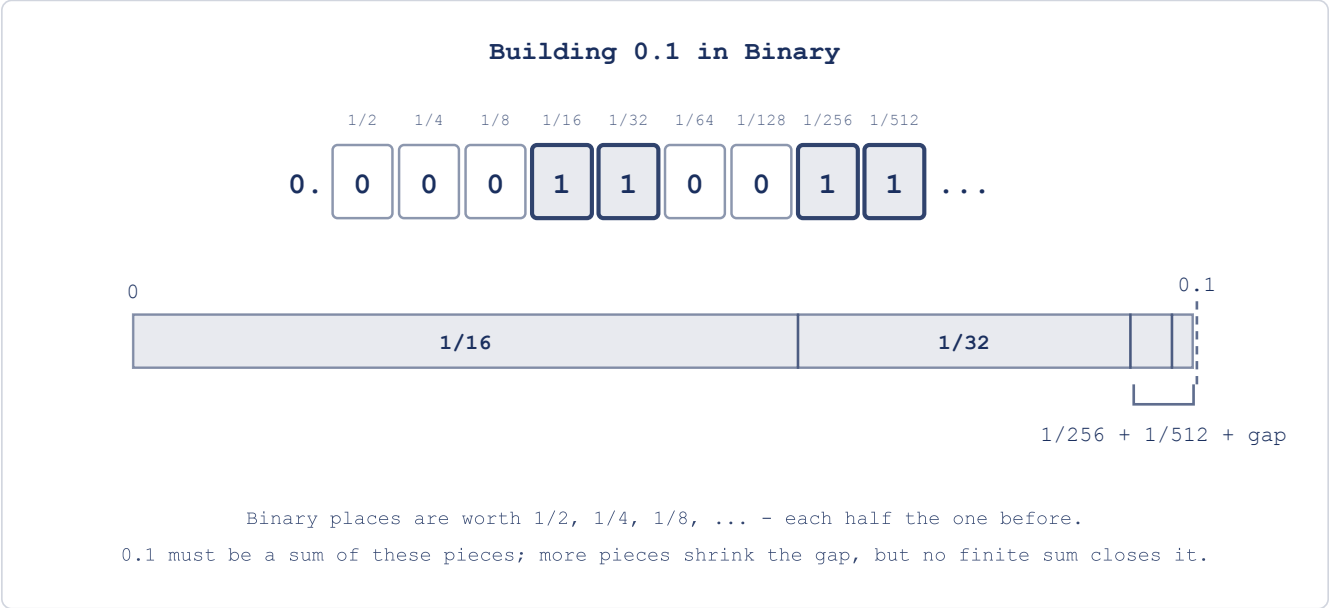


Figure 3.2: Building 0.1 in Binary

Written out as digits, that endless sum is 0.0001100110011…, the block 0011 repeating forever. A float has a fixed size - 64 bits in Python - so the summing has to stop, and what's stored is the closest value those pieces can build (the last bit gets rounded up rather than chopped, as it happens, which is why the stored value lands a hair above 0.1 instead of below). Those 64 bits are a memory budget: precision is being bought with space, and the error is what the budget can't cover. The same happens to 0.2. Add them, and the two tiny surpluses combine into something visibly different from 0.3; Python prints exactly what it's holding (the full storage format has a bit more machinery - a sign, an exponent - but you don't need it; the never-ending sum is the whole story).

This also tells you which decimals are safe: anything that is a finite sum of these pieces stores exactly. 0.5 is a single piece (1/2), 0.25 is another (1/4), 0.75 is two of them (1/2 + 1/4) - all perfect. 0.1, 0.2, and 0.3 never are.

For most purposes the error is far too small to matter, and later in this chapter `:.2f` will round it away before anyone sees it - the customer's receipt says $0.30. For financial systems where every cent must be exact, Python has a `decimal` module that stores base-10 digits directly; we won't need it in this book, but know it exists.

When the error does matter and nobody accounts for it, the consequences can be real. In 1991, during the Gulf War, an American Patriot missile battery in Dhahran, Saudi Arabia failed to track and intercept an incoming Iraqi Scud, which struck an army barracks and killed 28 soldiers. The investigation traced the failure to exactly the number on this page: the system's clock counted time as a whole number of tenths of a second - and whole numbers, as we just saw, store perfectly. But to use that count in tracking calculations, the software multiplied it by 0.1 to convert to seconds, and the system's 24-bit hardware could only hold a chopped-off version of 0.1's endless binary sum, a constant off by about 0.000000095 - more than ten billion times the error in Python's 64-bit floats, the price of the smaller budget. Multiplied by the clock count after 100 hours of continuous operation, the computed time had drifted by about a third of a second, and a Scud covers more than half a kilometer in that time: the radar looked for the missile in the wrong place, found nothing, and dismissed the alert. The Army already knew about the drift - a corrected version of the software arrived in Dhahran the day after the attack.

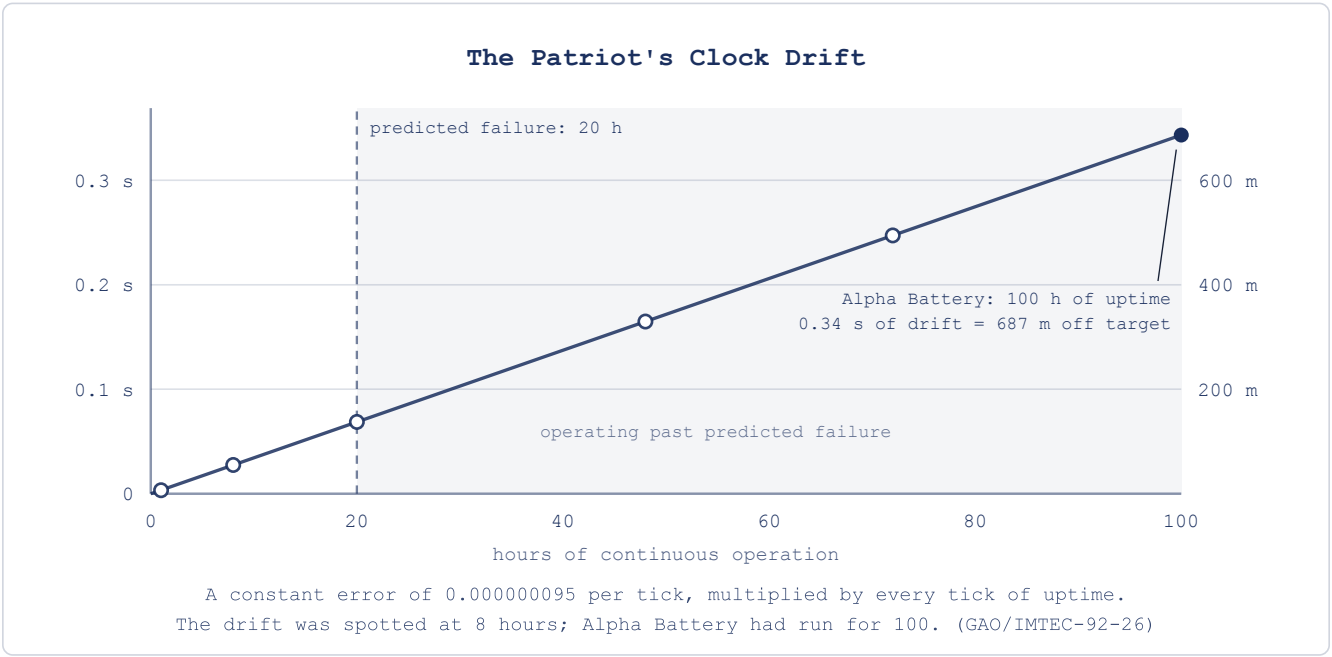


Figure 3.3: The Patriot's Clock Drift

That's the one leak in the float abstraction worth knowing about. It still doesn't solve the receipt problem, though - the crash wasn't about precision, it was about strings pretending to be numbers. To fix that, we need to turn strings into numbers.

3.5 Type Conversion

Python provides built-in functions that convert values from one type to another.

`int()` converts to an integer. It works on strings that contain whole numbers, and on floats, but with a catch worth pausing on: it *truncates*, it doesn't round. It chops off everything after the decimal point and keeps the whole-number part, so `int(3.99)` is `3`, not `4`. That's a real trap - use `int()` on a price expecting it to round to the nearest value and you'll silently lose the fractional part. When rounding is what you actually want, that's a different function, `round()`: `round(3.99)` gives `4`, and `round(3.14159, 2)` gives `3.14`. Keep the two straight - `int()` chops, `round()` rounds.

```
>>> int("42")
42
>>> int(3.99)
3
>>> round(3.99)
4
```

`float()` converts to a float. It works on strings that contain numbers (with or without decimal points) and on integers:

```
>>> float("14.99")
14.99
>>> float(42)
42.0
```

`str()` converts anything to a string:

```
>>> str(42)
'42'
>>> str(3.14)
'3.14'
```

Now we can fix the receipt. If `quantity` is the string `"2"` and `price` is the string `"14.99"`, then:

```
>>> int("2") * float("14.99")
29.98
```

That's the total, and it's exactly the `float(price) * int(quantity)` we promised at the end of Chapter 2: `int()` turns the string `"2"` into the integer `2`, `float()` turns `"14.99"` into the float `14.99`, and the multiplication works because both values are now numbers.

You can convert at the moment you get the input:

```
quantity = int(input("Quantity: "))
price = float(input("Price per item: "))
```

Or convert later when you need the values as numbers - either way, the conversion has to happen before you do math.

What happens if the user types something that can't be converted? Try `int("hello")` in the REPL. Python raises a `ValueError`, and now you know how to read the traceback it prints. We'll learn how to handle these errors gracefully in a later chapter; for now, we'll trust that the user enters valid numbers.

Type conversion solves the arithmetic problem, but the receipt also needs to display values in a specific format - dollar signs, two decimal places, aligned columns - and string concatenation with + gets clumsy fast. There's a better way as we'll see.

3.6 Formatting Output with F-Strings

An **f-string** (formatted string literal) lets you embed expressions directly inside a string. Prefix the string with `f` and put expressions inside curly braces:

```
name = "Hannah"
age = 30
print(f"Hello, my name is {name} and I am {age} years old.")
```

```
Hello, my name is Hannah and I am 30 years old.
```

Compare that to the concatenation approach: `"Hello, my name is " + name + " and I am " + str(age) + " years old."` The f-string is shorter, more readable, and doesn't require converting `age` to a string manually.

You can put any expression inside the braces, not just variable names:

```
x = 10
y = 20
print(f"The sum of {x} and {y} is {x + y}")
```

```
The sum of 10 and 20 is 30
```

F-strings also support format specifiers after a colon, and this is where they become essential for the receipt.

To control decimal places, use `:.Nf` where N is the number of digits after the decimal point:

```
price = 14.99
tax = price * 0.08
print(f"Tax: ${tax:.2f}")
```

```
Tax: $1.20
```

Without `:.2f`, the tax would display as `1.1992`, all four decimal places - and in larger calculations, the rounding error from earlier can surface as a long trail of digits. The format specifier rounds and fixes the display to exactly two decimal places, which is what you want for currency.

To add comma separators for large numbers, use `:,`:

```
population = 8000000
print(f"Population: {population:,}")
```

```
Population: 8,000,000
```

You can combine them: `${amount:,.2f}` gives you commas and two decimal places, which is exactly what currency formatting needs.

For alignment, use < (left), > (right), or ^ (center) with a width:

```
print(f"{'Item':<20}{'Price':>10}")
print(f"{'Book':<20}{'$14.99':>10}")
print(f"{'Notebook':<20}{'$3.50':>10}")
```

```
Item                      Price
Book                     $14.99
Notebook                  $3.50
```

This aligns item names to the left in a 20-character field and prices to the right in a 10-character field - that's how receipts get their clean columns. If you did the badge exercise in Chapter 2, you've seen centering before: `^` does the same job as the `.center()` method, but inside an f-string, where it can sit alongside other formatting.

Now we have types, conversions, and formatting - almost enough to rebuild the receipt. But the receipt takes text input from the user, and user input is messy: someone might type " Bookshop " with extra spaces, or "hannah" in lowercase when you want "Hannah" on the receipt. We need tools to clean up and reshape strings before displaying them.

3.7 Working with Strings

Strings have methods: built-in features or tools (actually functions like the ones for converting data types, print, input; we'll talk more about this later) you call with a dot after the string. You don't need to memorize all of them, but a handful show up constantly.

Case conversion: `"hello".upper()` gives `"HELLO"`, `"HELLO".lower()` gives `"hello"`, `"hello world".title()` gives `"Hello World"`. We'll use `.title()` in the receipt to normalize names regardless of how the user types them, and `.strip()` to remove accidental spaces.

Checking content: `"Hello".startswith("He")` returns `True`, `"Hello".endswith("lo")` returns `True`, `"fun" in "Python is fun"` returns `True`. These let you inspect strings without manually picking through characters.

Finding and replacing: `"Hello, world!".find("world")` returns `7` (the position where "world" starts), and `"Hello, world!".replace("world", "Python")` returns `"Hello, Python!"`. Note that `replace()` doesn't change the original string - it returns a new one. Strings in Python are **immutable**: once created, they can't be modified, and every operation that appears to change a string actually creates a new one.

Splitting and joining: `"apple,banana,cherry".split(",")` returns `['apple', 'banana', 'cherry']` (a list, which we'll cover properly in a later chapter), and `", ".join(['apple', 'banana', 'cherry'])` goes the other way: `"apple, banana, cherry"`. The split/join pair is how you take structured text apart and put it back together.

Stripping whitespace: `"  hello  ".strip()` returns `"hello"`, removing spaces (and tabs, newlines) from both ends; `lstrip()` and `rstrip()` remove from the left or right only. This is essential when processing user input, which often has accidental spaces.

Length: `len("hello")` returns `5`. You met this in the Chapter 2 postcard exercise - it's a built-in function, not a method, so you call it with the string as an argument rather than using dot notation.

Individual characters: `"hello"[0]` gives `"h"`, `"hello"[-1]` gives `"o"`. You used `name[0]` for the initials exercise in Chapter 2; now the full picture: Python uses zero-based indexing (the first character is at position 0, the second at position 1) and negative indices count from the end.

Slicing: `"hello"[1:4]` gives `"ell"` (characters at positions 1, 2, and 3 - the end index is excluded), `"hello"[:3]` gives `"hel"`, `"hello"[3:]` gives `"lo"`. This lets you extract portions of a string by position.

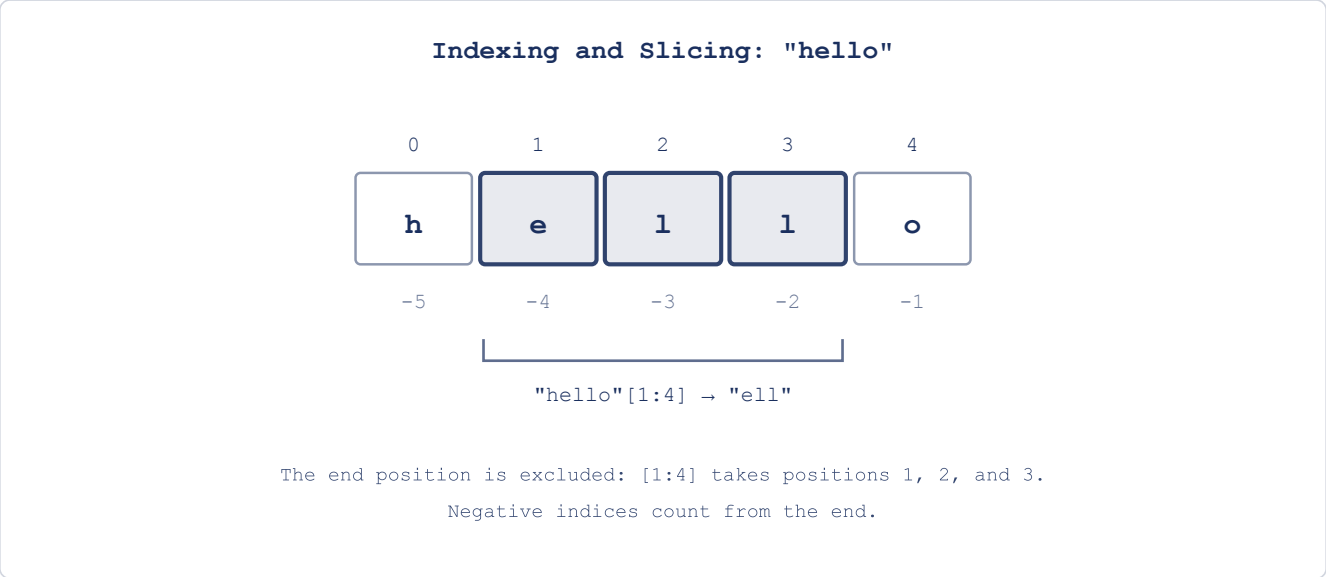


Figure 3.4: Indexing and Slicing

That's enough string tools to build something real. Let's go back to the receipt.

3.8 Hands-On: Receipt Printer, Completed

The receipt from Chapter 2 collected input and formatted it as text, but couldn't do any math, now we can fix that. We'll rebuild it with type conversions for arithmetic, string methods to clean up input, and f-strings for formatting. If you did the first exercise in Chapter 2, you already extended the receipt to three line items - that's exactly the skeleton we're rebuilding here, this time with working math.

Start a fresh project (keep your Chapter 2 folder as it is - we'll want to compare against it later):

```
uv init chapter3
cd chapter3
```

Create a file called `receipt.py`. Start with the input, converting numbers at the point of entry and cleaning up text with string methods. We're also moving the customer name up next to the store name: with three items coming, it's cleaner to collect all the header information first:

```
# receipt.py
print("=== Receipt Printer ===", end="\n\n")

store_name = input("Store name: ").strip().title()
customer_name = input("Customer name: ").strip().title()
print()

item1_name = input("Item 1 name: ").strip()
item1_qty = int(input("Item 1 quantity: "))
item1_price = float(input("Item 1 price: "))

item2_name = input("Item 2 name: ").strip()
item2_qty = int(input("Item 2 quantity: "))
item2_price = float(input("Item 2 price: "))

item3_name = input("Item 3 name: ").strip()
item3_qty = int(input("Item 3 quantity: "))
item3_price = float(input("Item 3 price: "))
```

Notice what changed from Chapter 2. Quantities go through `int()` and prices through `float()` - they're numbers now, not strings, which means we can compute line totals and a grand total. The store and customer names get `.strip()` (removing accidental spaces) and `.title()` (capitalizing properly), so even if the user types " bookshop ", the receipt prints"Bookshop." Item names get `.strip()` too.

Now the calculations:

```
# receipt.py
# ...existing code above

item1_total = item1_qty * item1_price
item2_total = item2_qty * item2_price
item3_total = item3_qty * item3_price

subtotal = item1_total + item2_total + item3_total
tax_rate = 0.08
tax = subtotal * tax_rate
total = subtotal + tax
```

Just six lines of arithmetic - in Chapter 2 this was impossible, now it works because the values are the right types.

Finally, the formatted output, we'll use f-strings with alignment to make clean columns. To get the dollar sign and the number to align as a unit, we format each price as a string first, then right-align that string in a fixed-width field:

```
# receipt.py
# ...existing code above

width = 35

print()
```

```
print("=" * width)
print(f"{store_name:^{width}}")
print("=" * width)
print(f"Customer: {customer_name}")
print("-" * width)

print(f"{'Item':<15} {'Qty':>3} {'Price':>7} {'Total':>7}")
print("-" * width)

p1 = f"${item1_price:.2f}"
t1 = f"${item1_total:.2f}"
print(f"{item1_name:<15} {item1_qty:>3} {p1:>7} {t1:>7}")

p2 = f"${item2_price:.2f}"
t2 = f"${item2_total:.2f}"
print(f"{item2_name:<15} {item2_qty:>3} {p2:>7} {t2:>7}")

p3 = f"${item3_price:.2f}"
t3 = f"${item3_total:.2f}"
print(f"{item3_name:<15} {item3_qty:>3} {p3:>7} {t3:>7}")

print("-" * width)
sub_str = f"${subtotal:.2f}"
tax_str = f"${tax:.2f}"
total_str = f"${total:.2f}"
print(f"{'Subtotal:':>27} {sub_str:>7}")
print(f"{'Tax (8%):':>27} {tax_str:>7}")
print("=" * width)
print(f"{'TOTAL:':>27} {total_str:>7}")
print("=" * width)
print(f"{'Thank you for your purchase!':^{width}}")
```

Run it with `uv run receipt.py` (or `uv run python receipt.py` - both work; `uv run` will hand a `.py` file straight to Python either way). Enter "bookshop" as the store and "hannah" as the customer (both lowercase, to see `.title()` at work), then three items: "Python Book" at quantity 2, price 29.99; "Notebook" at quantity 3, price 4.50; "Pen" at quantity 5, price 1.25. The output looks like:

```
====================================
             Bookshop
====================================
Customer: Hannah
------------------------------------
Item            Qty   Price   Total
------------------------------------
Python Book       2  $29.99  $59.98
Notebook          3   $4.50  $13.50
Pen               5   $1.25   $6.25
------------------------------------
                  Subtotal:  $79.73
                  Tax (8%):   $6.38
====================================
```

```
                      TOTAL:  $86.11
==================================
   Thank you for your purchase!
```

Compare this to the Chapter 2 version: that receipt was a string-formatting exercise - it could display prices but not add them; this receipt computes real totals, applies tax, cleans up input, and formats everything with aligned columns and two decimal places. The difference is type conversion (turning input strings into numbers), string methods (cleaning and normalizing text), and f-strings (controlling how values display).

The binary limitation from earlier is at work here too: the stored subtotal is actually `79.72999999999999`, and the total is `86.10839999999999`. But `:.2f` rounds for display, so the customer sees `$79.73` and `$86.11`.

Look at how the columns work: each price gets formatted as a string with a dollar sign first (`f"${item1_price:.2f}"` produces `"$29.99"`), then that string gets right-aligned in a 7-character field (`{p1:>7}` produces `"  $29.99"` or `"   $4.50"`); since the header uses the same field widths (`{'Price':>7}`), everything lines up. The two-step approach - format the value, then align the formatted string - is a pattern you'll use whenever you need columns with mixed content like currency symbols.

Try experimenting: change the tax rate, add a discount line. What happens if an item name is longer than 15 characters? What happens if a price is over $999.99 (it would need more than 7 characters)? The format specifiers control the layout, and testing their limits teaches you what they can and can't handle.

The receipt works, but there's an obvious problem: we hardcoded three items. What if the customer buys two items? What if they buy ten? Right now you'd need a separate set of variables for each item, and you'd need to know the count before writing the code. In Chapter 4, we'll learn how to make decisions (what if the quantity is zero?) and in Chapter 5, how to repeat actions (process as many items as the customer has).

3.9 Chapter Summary

The receipt crashed in Chapter 2 because `input()` returns strings, and strings can't do arithmetic; the traceback told us exactly that, once we learned to read it. The fix was understanding types: Python distinguishes between integers, floats, strings, and booleans, and the type of a value determines what operations it supports. The same operator behaves differently depending on the types involved - `+` adds numbers but concatenates strings, `*` multiplies numbers but repeats strings. And floats are binary approximations: 0.1 has no exact binary form, so 0.1 + 0.2 carries an error in the sixteenth decimal place, but it's mostly harmless once display rounding is in place.

Type conversion helps us: `int()`, `float()`, and `str()` let you move values between types when you need to: `int(input("Quantity: "))` turns the user's text into a number you can compute with. F-strings handle the other direction, embedding numbers inside formatted text with control over decimal places, alignment, and width.

Strings are immutable sequences of characters with built-in methods for case conversion, searching, replacing, splitting, and stripping whitespace. You access individual characters by index and extract substrings by slicing - tools that show up constantly once you start processing text.

The rebuilt receipt computes real totals with tax and formatted columns, and cleans up user input with `.strip()` and `.title()`, but it handles exactly three items, hardcoded. What if the customer has more? What if they have fewer? The program can't adapt because it follows the same path every time. In the next chapter, we'll introduce conditionals: the ability to make decisions based on data, so your programs can respond to different situations differently.

3.10 Exercises

1. The receipt uses a fixed tax rate of 8%. Modify the program to ask the user for the tax rate as a percentage (like `8.5`), convert it to a decimal, and apply it. Then add a second version of the total that shows what the bill would be with no tax. Print both totals side by side. How does the formatting need to change when the tax rate has a decimal place?
2. Write a program that asks for a distance in kilometers and converts it to miles, feet, and inches (1 km = 0.621371 miles, 1 mile = 5280 feet, 1 foot = 12 inches). Display all four values aligned in a column with appropriate precision: kilometers and miles to two decimal places, feet to one, inches to zero. What happens to the inches value when you enter a very small distance like 0.001 km?
3. Write a program that asks for a full name (like "jane doe") and prints it in four formats: all uppercase, all lowercase, title case, and as initials with periods (like "J.D."). You'll need `split()` to separate the words and indexing (`word[0]`) to get first characters. What happens if the user enters a name with three parts, like "Mary Jane Watson"? What about a single name with no space?
4. Write a program that asks the user for a sentence and reports: the number of characters, the number of words (use `split()`), the first word, the last word, and the sentence reversed (use slicing with `[::-1]`). The reversed sentence will look like gibberish, but compare it to the original: is a palindrome sentence (one that reads the same forwards and backwards) possible with whole words? Try entering "was it a car or a cat i saw" and check.
5. F-strings let you create formatted tables. Write a program that asks for three students' names and their scores on two tests. Display the results as a table with right-aligned scores, each student's average to one decimal place, and a class average at the bottom. The tricky part is getting the columns to line up. What's the minimum field width that keeps the table readable for any reasonable name length?
6. Write a program that asks for a price in dollars (like `1234.56`) and prints it formatted three ways: plain (`1234.56`), with commas (`1,234.56`), and as a full currency string (`$1,234.56`). Then ask for a second price and display both prices right-aligned in a 15-character field, one above the other, with a line of dashes between them and a total below. This mimics the column-alignment logic from the receipt but requires you to figure out the format specifiers yourself.

Downey, Allen B. 2015. *Think Python: How to Think Like a Computer Scientist*. 2nd ed. O'Reilly Media.

Guttag, John. 2016. *Introduction to Computation and Programming Using Python: With Application to Understanding Data*. 2nd ed. MIT Press.

Hammack, Richard H. 2020. *Book of Proof*. 3rd ed. Virginia Commonwealth University.

Percival, Harry J. W., and Bob Gregory. 2020. *Architecture Patterns with Python: Enabling Test-Driven Development, Domain-Driven Design, and Event-Driven Microservices*. O'Reilly Media.

Pólya, George. 2014. *How to Solve It: A New Aspect of Mathematical Method*. 2nd ed. Princeton University Press.

Ramalho, Luciano. 2021. *Fluent Python: Clear, Concise, and Effective Programming*. 2nd ed. O'Reilly Media.

Robson, Patrick. 2021. *Robust Python: Write Clean and Maintainable Code*. O'Reilly Media.

Shaw, Zed A. 2013. *Learn Python the Hard Way: A Very Simple Introduction to the Terrifyingly Beautiful World of Computers and Code*. 3rd ed. Zed Shaw's Hard Way Series. Addison-Wesley Professional.

Slatkin, Brett. 2019. *Effective Python: 125 Specific Ways to Write Better Python*. 2nd ed. Effective Software Development Series. Addison-Wesley Professional.

Sweigart, Al. 2019. *Automate the Boring Stuff with Python: Practical Programming for Total Beginners*. 2nd ed. No Starch Press.

Velleman, Daniel J. 2019. *How to Prove It: A Structured Approach*. 3rd ed. Cambridge University Press.